nidavellir-devices

nidavellir devices system

high level design

四个层次如下：

```
        instances//以产品或硬件board为名
    applications drivers  //应用和公共驱动服务
        nidavellir//开发工具包
        packages //组件包
```

packages

平台组件 或者叫 包

基础服务层

domain领域层

含业务概念/业务状态/业务规则

不可单独部署

不可单独执行

导出`packages.h`供上层`application`使用

每个组件的配置

编程注意事项:

1. 多数包能够使用到多个环境(前后台/多线程/多进程/动态/静态/dll/lib)下，故单个包不能依赖别的包，只依赖fifo和消息。更不能直接操作设备(外设和内设)，特别是业务包不能依赖操作系统包。
2. 同理，包要满足**可重入和多实例**的要求
3. 消息格式设计 'cmd len data cmd'
4. 此目录单独代码仓库

applications

调用packages构建的业务强相关的应用

三种形态: `baremetal/thread/process`,分别对应 `baremetal_entry.c;thread_entry.c;process_entry.c`。

> 如何将单个application方便地"打包"成thread and process?

可单独执行

可单独部署

独立仓库

导出 applications.h

编程模型

总体上，采用消息或事件驱动模型。其中，消息队列的设计如下

- 在前后台环境下，利用`fifo`来充当消息队列。
- 在多线程和多进程环境下，利用操作系统提供的线程间通信和同步机制(信号量/事件标志/队列/邮箱/管道)来充当消息队列，提高消息处理速度。

具体上，创建application可分以下步骤

1. 在applications目录下新建应用目录
2. 在应用目录中新建baremetal_entry.c
3. baremetal_entry.c的基本模型如下

```
int app_entry(struct instance_t* instance,
              struct pipeline_t* pipeline, struct message_t* mesg)
{
    int r = 0;
    //1.接入校验，验证接收到的消息的完整性和合法性
    //2.处理消息
    switch (mesg->ctrl)
    {
    case 0x__:
        //3.调用领域层完成业务  反映用例场景
        break;
    case 0x__:
        //3.调用领域层完成业务  反映用例场景
        break;
    default:
        break;
    }

    return 0;
}
```

instances

产品实例集合

里面的每个实例对应一个产品

独立仓库

其包括以下内容:

1. 硬件配置声明: 当前实例所使用的硬件和设备配置清单
2. 数据资源声明: 当前实例所使用的"数据声明"及其在硬件上的布局情况

   数据资源的声明方法？

   数据资源的管理方法？
3. 组件配置声明: 当前实例所使用的组件及其配置
4. 应用配置声明: 当前实例所使用的应用及其配置
5. 部署方式声明: `baremetal/multi_threads/multi-processes`

本目录下只有一个函数入口`xxxx_instance_entry`. **即要得到不同的instance，只需改写此入口函数。**

例如:

```
int baremetal_instance_entry(int argc, char** argv)
{
    //构造instance
        //初始化硬件
        //初始化固件
        //初始化资源管理器
            //获取设备信息
            //获取数据模型布局
            //绑定模型  设备 和方法
        //初始化软件
            //创建每个application
            //初始化每个application
            //每个application的control block如何初始化?且如何绑定到instance里?
            //每个application如何访问自己的control block?
        //链接和绑定硬件 固件 和 软件
    //启动主循环，执行此instance
    return baremetal_loop(instance);
}
```